

2110452 HIGH PERFORMANCE ARCHITECTURE

Homework 3: Thread-Level Parallelism

Computer Engineering, Chulalongkorn University

Worralop Srichainont

March 9, 2026

Contents

1	Problem I	1
2	Problem II	1
2.1	No OpenMP	2
2.2	OpenMP	3
2.3	Run Experiment	4
2.4	Result	4
3	Problem III	5

1 Problem I

What is the main benefit of OpenMP over standard threads (e.g. POSIX threads)?

- **Code Simplicity:** To utilize OpenMP is lot easier than the standard threads. You need to just write the simple compiler directive (e.g. `#pragma omp parallel`).
- **No Manual Management:** The standard threads such as POSIX threads need to manually manage the thread function. You need to manually initialize thread arrays by using `pthread_create` or synchronizing thread by using `pthread_join`. But in OpenMP, you can just write the compiler directive to perform thread parallelization.

2 Problem II

For a given code, modify it to take the benefit of simultaneous multithreading processor using OpenMP. With the new code, what is the potential speed up?

Problem II

```
1 #include<stdio.h>
2
3 #define ARR_SIZE 100000
4
5 int a[ARR_SIZE];
6
7 int main(void) {
8     for (int i = 0; i < ARR_SIZE; i++) {
9         a[i] = (2 * i) + i;
10        printf("a[%d], %d\n", i, a[i]);
11    }
12    return 0;
13 }
```

The modified code is in the next page →

2.1 No OpenMP

This code is modified to be able to measure the runtime for 10 iterations. To measure the runtime, simply use `clock()` command from `time.h`.

Problem II - No OpenMP

```
1 #include<stdio.h>
2 #include<time.h>
3
4 #define TEST_ITERATIONS 10
5 #define ARR_SIZE 1000
6
7 double run_test();
8
9 int main(void) {
10     // Store test runtime
11     double test_runtime[TEST_ITERATIONS];
12     for (int i = 0; i < TEST_ITERATIONS; i++) {
13         test_runtime[i] = run_test();
14     }
15
16     // Display runtime
17     printf("===== No OpenMP Runtime =====\n");
18     for(int i = 0; i < TEST_ITERATIONS; i++) {
19         printf("TEST %d: %f s\n", i, test_runtime[i]);
20     }
21     printf("===== \n");
22     return 0;
23 }
24
25 double run_test() {
26     int a[ARR_SIZE];
27
28     // Run an experiment
29     clock_t begin_time = clock();
30     for (int i = 0; i < ARR_SIZE; i++) {
31         a[i] = (2 * i) + i;
32         printf("a[%d], %d\n", i, a[i]);
33     }
34     clock_t end_time = clock();
35
36     // Return executed time
37     double run_time = ((double) end_time - begin_time) / CLOCKS_PER_SEC;
38     return run_time;
39 }
```

2.2 OpenMP

This code is modified to be able to measure the runtime for 10 iterations. To perform thread-level parallelization on the for loop, just call `#pragma omp parallel for` and to measure the runtime, simply use `omp_get_wtime()` command from `omp.h`.

Problem II - OpenMP

```
1 #include<stdio.h>
2 #include<omp.h>
3
4 #define TEST_ITERATIONS 10
5 #define ARR_SIZE 1000
6
7 double run_test();
8
9 int main(void) {
10     // Store test runtime
11     double test_runtime[TEST_ITERATIONS];
12     for (int i = 0; i < TEST_ITERATIONS; i++) {
13         test_runtime[i] = run_test();
14     }
15
16     // Display runtime
17     printf("===== OpenMP Runtime =====\n");
18     for(int i = 0; i < TEST_ITERATIONS; i++) {
19         printf("TEST %d: %f s\n", i, test_runtime[i]);
20     }
21     printf("===== \n");
22     return 0;
23 }
24
25 double run_test() {
26     int a[ARR_SIZE];
27
28     // Run an experiment
29     double begin_time = omp_get_wtime();
30     #pragma omp parallel for
31     for (int i = 0; i < ARR_SIZE; i++) {
32         a[i] = (2 * i) + i;
33         printf("a[%d], %d\n", i, a[i]);
34     }
35     double end_time = omp_get_wtime();
36
37     // Return executed time
38     double run_time = end_time - begin_time;
39     return run_time;
40 }
```

2.3 Run Experiment

By running both C files at the same time, we can write a batch script.

- We are using GCC compiler to compile the C program into .exe file.
- For opm.c file, use -fopenmp to allow compiler to perform thread parallelization.

Batch Script

```
1 REM COMPILE PROGRAM USING GCC COMPILER
2 gcc no_omp.c -o no_omp.exe
3 gcc -fopenmp omp.c -o omp.exe
4
5 REM EXECUTE EXPERIMENT
6 no_omp.exe
7 PAUSE
8
9 omp.exe
10 PAUSE
```

2.4 Result

After the running the batch script, the program shows the following result.

No.	No OpenMP	OpenMP
1	0.729 s	2.400 s
2	0.681 s	2.542 s
3	2.485 s	2.020 s
4	2.084 s	1.907 s
5	2.342 s	2.826 s
6	5.136 s	2.107 s
7	3.029 s	1.717 s
8	2.375 s	1.749 s
9	2.898 s	2.351 s
10	3.177 s	1.941 s
Trimmed mean	2.390 s	2.127 s

Table 1: Experiment Result

From the trimmed mean of both version, we can easily calculate the speedup of the thread as follows:

$$\text{Speedup} = \frac{\text{Time}_{\text{No OpenMP}}}{\text{Time}_{\text{OpenMP}}}$$

$$\text{Speedup} = \frac{2.390 \text{ s}}{2.127 \text{ s}}$$

$$\text{Speedup} \approx 1.124$$

The experiment shows that the thread-level parallelization in C language is **1.124** times faster.

3 Problem III

Base on OpenMP, explain the concepts of work sharing constructs for loop constructs, sections, single and workshare.

- **loop constructs: for and do:** This construct distributes the iterations of a loop across the available threads in a parallel region so that they execute concurrently.
- **sections:** This breaks down work into blocks of code. Each section is executed once by a thread. This allows different thread to perform different task simultaneously.
- **single:** This construct specifies that a particular block of code inside a parallel region should be executed by only one thread. The other threads will wait at the end of the block unless a `nowait` clause is used.
- **workshare:** This construct divides the execution of array syntax, `FORALL`, and `WHERE` statements among the threads, acting as an analog to the loop construct but for array operations.